

TREE TRAVERSAL

PROGRAM CODING:

```
#include<stdio.h>

#include<malloc.h>

struct node

{

    int value;

    struct node*left,*right;

};

struct node*root,*new1;

void insert(struct node*,struct node*);

void preorder(struct node*);

void inorder(struct node*);

void postorder(struct node*);

int main()

{

    int ch,v;

    clrscr();

    root=(struct node*)malloc(sizeof(struct node));

    printf("Enter the value of root:");

    scanf("%d",&root->value);

    root->left=root->right=NULL;

    printf("\n\t.....TREE TRAVERSAL....\n\t");

    printf("\n\t1.insert\n\t2.preorder\n\t3.postorder\n\t4.inorder\n\t5.exit");

    while(1)
```

```

{
printf("\n enter your option:");

scanf("%d",&ch);

switch(ch)
{
case 1:

new l=(struct node*)malloc(sizeof(struct node));

printf("Enter the value to be inserted\t");

scanf("%d",&new l->value);

new l->left=new l->right=NULL;

insert(root,new l);

break;

case 2:

printf("\n pre order traversal:");

preorder(root);

break;

case 3:

printf("\n post order traversal:");

postorder(root);

break;

case 4:

printf("\n in order traversal:");

inorder(root);

break;

case 5:

exit(0);

break;

```

```
    }  
}  
}
```

```
void preorder(struct node*root)  
{  
    if(root!=NULL)  
    {  
        printf("\n%d",root->value);  
        preorder(root->left);  
        preorder(root->right);  
    }  
}
```

```
void inorder(struct node*root)  
{  
    if(root!=NULL)  
    {  
        inorder(root->left);  
        printf("\n%d",root->value);  
        inorder(root->right);  
    }  
}
```

```
void postorder(struct node*root)  
{  
    if(root!=NULL)
```

```

    {
        postorder(root->left);
        postorder(root->right);
        printf("\n%d",root->value);
    }
}

void insert(struct node *root,struct node* new1)
{
    if(new1->value<root->value)
    {
        if(root->left==NULL)
            root->left=new1;
        else
            insert(root->left,new1);
    }
    if(new1->value>root->value)
    {
        I    f(root->right==NULL)
            root->right=new1;
        else
            insert(root->right,new1);
    }
}

```